

PISSIR

RELAZIONE FINALE

Connected Games Platform

PlayConnect

Piattaforma per collegare giochi fisici tradizionali a Internet

Anno accademico 2025/2026

Docente: Alberto Testa

Studenti / Gruppo: _____

Indice

1. Introduzione
 2. Specifiche funzionali
 3. Analisi tecnologica
 4. Scelta dell'approccio
 5. Architettura
 6. Implementazione
 7. Validazione
 8. Sicurezza e scelte didattiche
 9. Conclusione
- Appendice A - Casi d'uso testuali
- Appendice B - Matrice dei requisiti
- Appendice C - Procedura demo

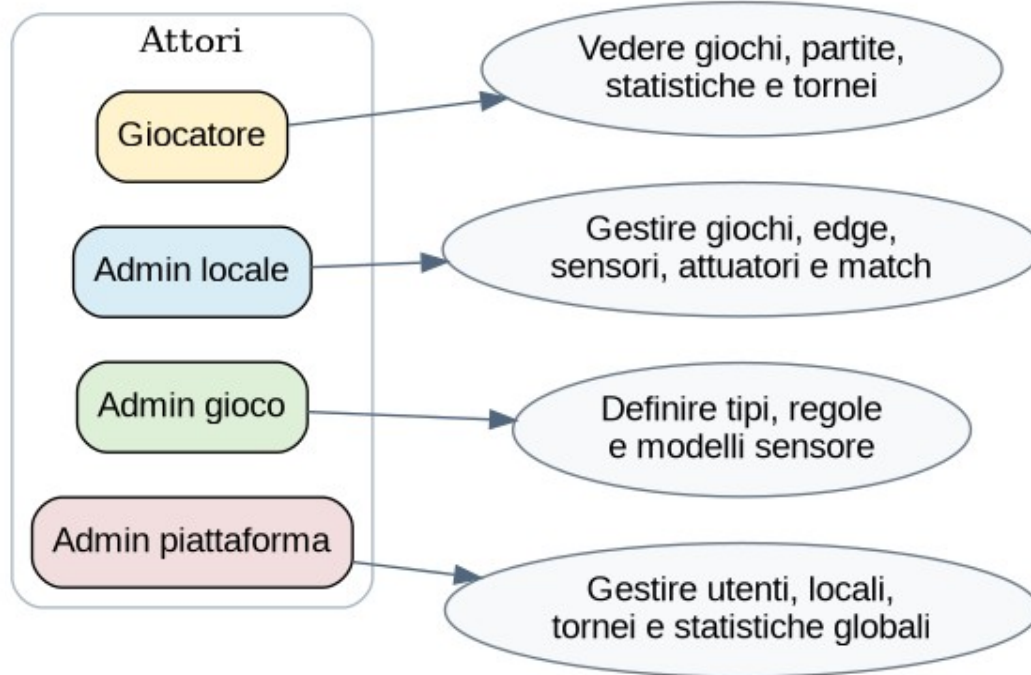
1. Introduzione

PlayConnect è una piattaforma che collega a Internet giochi fisici tradizionali, per esempio calciobalilla, freccette, bocce e Monopoli. I giochi continuano a essere usati normalmente, ma alcuni sensori rilevano eventi importanti. Un dispositivo edge installato nel locale riceve questi eventi e li invia al server tramite MQTT.

Il server salva partite, punteggi e statistiche. Gli utenti possono vedere i giochi disponibili, le proprie partite, i tornei e le classifiche. Gli amministratori possono configurare locali, giochi, sensori e dispositivi.

Il progetto è stato realizzato con codice volutamente semplice, diviso in piccoli moduli facili da spiegare durante l'esame.

2. Specifiche funzionali



Schema sintetico dei casi d'uso

2.1 Utenti

Il sistema gestisce quattro ruoli:

1. **Giocatore**
 - vede i giochi online;
 - consulta le proprie partite;
 - consulta statistiche e classifiche;
 - partecipa a tornei individuali o come membro di una squadra.
2. **Amministratore del locale**

- gestisce i giochi del proprio locale;
 - crea giocatori del proprio locale;
 - configura dispositivi edge, sensori e attuatori;
 - avvia partite individuali o a squadre;
 - controlla partite e statistiche locali.
3. **Amministratore del gioco**
- definisce nuovi tipi di gioco;
 - specifica gli eventi di inizio, punteggio e fine;
 - crea modelli di sensore associati ai tipi di gioco;
 - controlla le configurazioni di sensori e attuatori installati.
4. **Amministratore della piattaforma**
- gestisce utenti e locali;
 - crea amministratori locali e amministratori del gioco;
 - crea tornei;
 - seleziona i locali coinvolti nei tornei;
 - controlla statistiche globali.

2.2 Giochi e sensori

Ogni gioco appartiene a un locale e a un tipo di gioco. Un tipo di gioco contiene regole semplici:

- evento di inizio;
- evento punto del partecipante 1;
- evento punto del partecipante 2;
- evento di fine;
- eventuale limite di punteggio;
- possibilità di giocare a squadre.

I sensori possono essere fisici oppure simulati. Ogni sensore ha un evento associato e un topic MQTT.

2.3 Dispositivo edge

Il dispositivo edge:

- coordina i giochi del locale;
- riceve o simula gli eventi dei sensori;
- pubblica gli eventi su MQTT;
- mantiene una coda JSON quando la connessione non è disponibile;
- sincronizza la coda quando torna online;
- riceve comandi per gli attuatori;
- offre una piccola interfaccia locale sulla porta 8090.

2.4 Attuatori

Sono stati aggiunti attuatori simulati:

- display del punteggio;
- luce LED;
- eventuale buzzer.

Lo stato dell'attuatore viene aggiornato quando la partita inizia, quando cambia il punteggio e quando termina.

2.5 Partite

Una partita può essere:

- **INDIVIDUAL:** due giocatori registrati;
- **TEAM:** due squadre registrate.

Il server registra:

- gioco e locale;
- partecipanti;
- punteggio;
- vincitore;
- stato;
- data di inizio e fine;
- lista ordinata degli eventi.

2.6 Tornei

Un torneo:

- riguarda un solo tipo di gioco;
 - può coinvolgere uno o più locali;
 - è individuale oppure a squadre;
 - contiene partite divise per turno;
 - può avere una data prevista per ogni partita;
 - produce una classifica con 3 punti per vittoria e 1 per pareggio.
-

3. Analisi tecnologica

3.1 Tecnologie possibili per la comunicazione

REST diretto dal dispositivo

Il dispositivo edge potrebbe inviare ogni evento con una richiesta HTTP REST.

Vantaggi: semplice da capire e da provare con Postman.

Svantaggi: il dispositivo deve conoscere direttamente il server, ogni evento apre una richiesta e la gestione delle disconnessioni è meno naturale.

WebSocket

WebSocket mantiene una connessione aperta tra client e server.

Vantaggi: comunicazione bidirezionale e veloce.

Svantaggi: gestione più complessa della riconnessione e meno adatta a tanti piccoli dispositivi IoT.

Server-Sent Events

SSE permette al server di inviare aggiornamenti al browser.

Vantaggi: utile per aggiornare una pagina live.

Svantaggi: la comunicazione è principalmente server-verso-client e non sostituisce il canale dei sensori.

MQTT con message broker

MQTT usa topic e un broker centrale. Il dispositivo pubblica un messaggio senza conoscere direttamente chi lo userà.

Vantaggi: leggero, adatto a sensori, disaccoppia edge e server, supporta QoS e riconnessione.

Svantaggi: richiede un broker e bisogna definire bene i topic.

3.2 Installazione in rete privata

Il dispositivo edge può trovarsi dietro un router privato. Non è necessario aprire porte in ingresso, perché è il dispositivo a creare una connessione in uscita verso il broker MQTT.

Questo evita configurazioni NAT o port forwarding nel locale.

3.3 Accesso dalla rete pubblica

Il browser accede al frontend e all'API gateway. In una pubblicazione reale si userebbero:

- un dominio;
- HTTPS;
- un reverse proxy;
- autenticazione con token.

Nella demo locale Docker, il frontend è disponibile su `http://localhost:8080` e il gateway su `http://localhost:3000`.

3.4 Traffico e tempo reale

Gli eventi sono piccoli messaggi JSON, normalmente inferiori a 1 KB. Una partita produce pochi eventi al secondo. Il progetto usa MQTT QoS 1, quindi il messaggio viene consegnato almeno una volta. Per evitare che un evento ripetuto aumenti due volte il punteggio, ogni evento contiene un `event_uuid` unico.

Non è richiesto hard real-time. Un ritardo di alcuni secondi è accettabile per aggiornare il display e la pagina web.

4. Scelta dell'approccio

È stato scelto MQTT per la comunicazione tra edge e server.

La scelta è adatta perché:

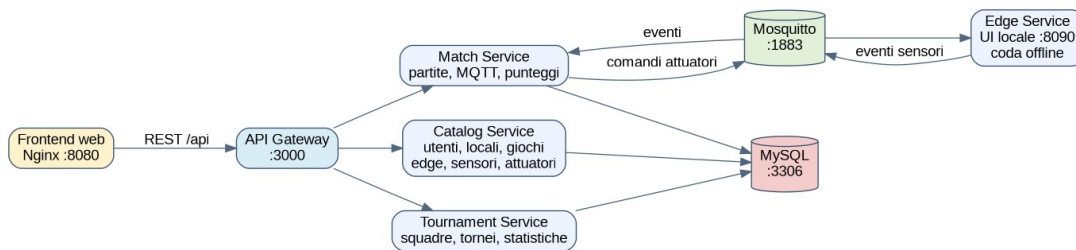
- i sensori inviano messaggi piccoli;
- il dispositivo può perdere temporaneamente la connessione;

- il broker separa chi produce gli eventi da chi li elabora;
- e possibile aggiungere altri servizi che ascoltano gli stessi topic;
- la struttura dei topic identifica locale, gioco e partita.

REST viene comunque usato tra interfaccia web e server. In questo modo ogni tecnologia viene usata per il compito più semplice:

- **REST:** gestione di utenti, giochi, tornei e consultazione dati;
- **MQTT:** eventi dei sensori e comandi degli attuatori.

5. Architettura



Architettura a microservizi

5.1 Componenti

Componente	Porta	Input	Output	Business logic
Frontend Nginx	8080	azioni utente	pagine HTML	interfaccia per ruoli
API Gateway	3000	richieste /api	risposta del servizio	instradamento verso il microservizio corretto
Catalog Service	3001	REST	JSON/MySQL	utenti, locali, tipi di gioco, giochi, edge, sensori, attuatori
Match Service	3002	REST + MQTT	JSON + comandi MQTT	partite, punteggi, eventi, deduplicazione
Tournament Service	3003	REST	JSON/MySQL	squadre, tornei, calendario, statistiche e classifica
Edge Service	8090	REST locale + sensori	MQTT	simulazione sensori, coda offline, attuatori locali
Mosquitto	1883	messaggi MQTT	messaggi MQTT	broker dei messaggi
MySQL	3306	query dei servizi server	dati persistenti	database centrale

5.2 Perché sono microservizi

I tre servizi applicativi sono eseguiti in container separati. Ogni servizio espone solo le API del proprio ambito. Il gateway nasconde questa divisione al frontend.

Il database è condiviso per mantenere il progetto didattico semplice. Solo i componenti server accedono direttamente a MySQL; frontend ed edge non accedono mai al database.

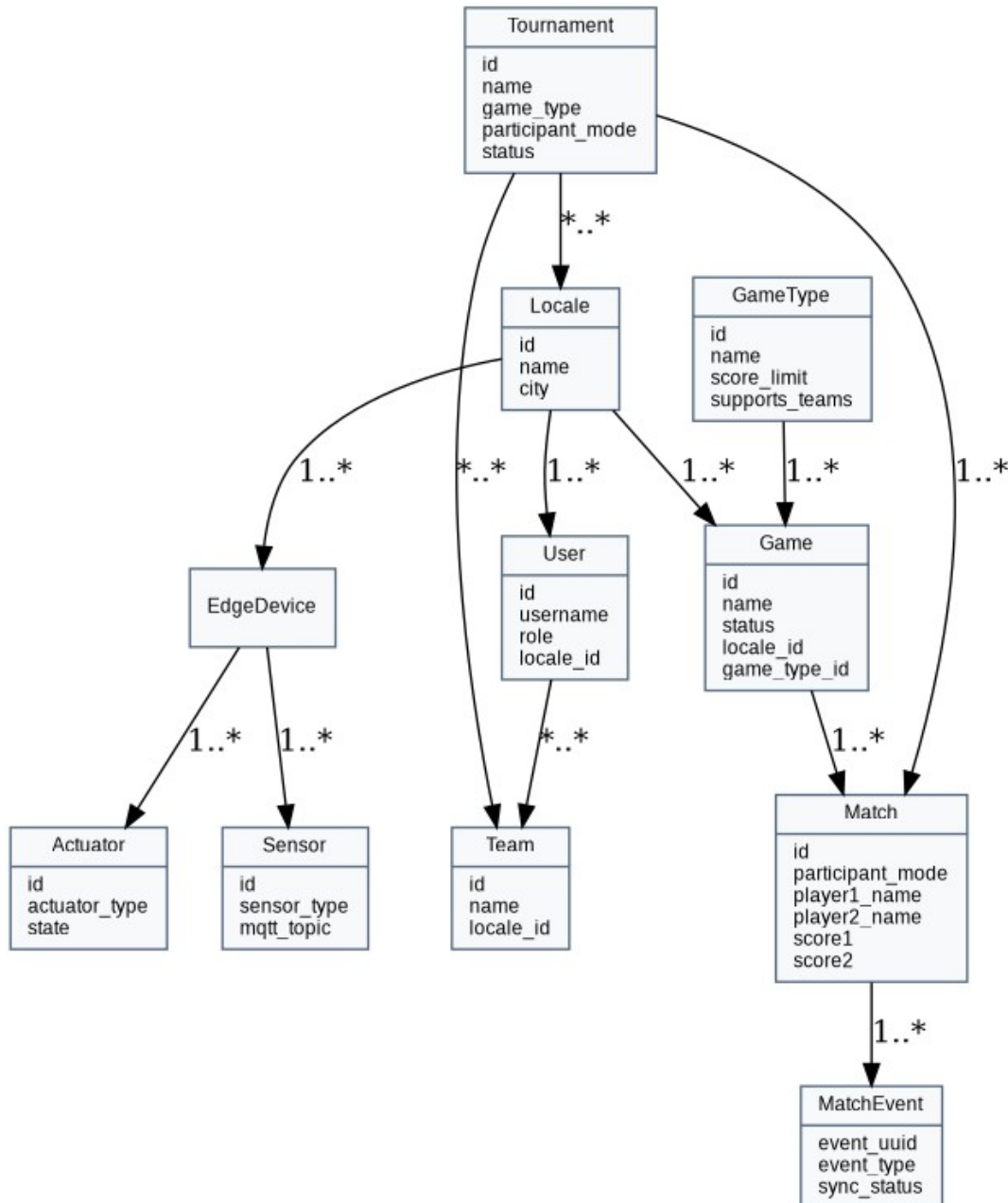
5.3 Attività concorrenti

Nel progetto sono presenti più attività che lavorano contemporaneamente:

- Express riceve richieste REST;
- il Match Service ascolta i topic MQTT;
- il client MQTT tenta la riconnessione;
- l'Edge Service invia heartbeat ogni 5 secondi;
- l'Edge Service sincronizza la coda offline;
- il frontend aggiorna pagine live con polling;
- una simulazione partita genera eventi con timer.

Node.js usa un event loop. Non viene creato manualmente un thread per ogni richiesta. Le operazioni di rete sono asincrone e non bloccano l'interfaccia.

6. Implementazione



Principali entità del dominio

6.1 Moduli principali

Controller

I controller ricevono la richiesta, controllano ruolo e dati e chiamano i servizi.

Esempi:

- `gameTypeController.js` gestisce tipi di gioco e modelli sensore;

- deviceController.js gestisce edge, sensori e attuatori;
- matchController.js avvia partite e controlla gli accessi;
- tournamentController.js gestisce locali, squadre e calendario del torneo.

Service

I service contengono la logica che può essere riutilizzata:

- matchEventService.js aggiorna punteggi e salva eventi;
- actuatorService.js aggiorna display e LED;
- tournamentService.js costruisce classifica e collegamenti.

Utility

Le utility contengono regole semplici e facilmente testabili:

- autorizzazioni;
- validazione dei dati;
- calcolo della classifica;
- controllo compatibilità partita-torneo.

6.2 Interfaccia utente

Sono presenti pagine diverse per ogni ruolo.

- Piattaforma: dashboard, locali, utenti, squadre, statistiche, tornei.
- Locale: dashboard, giochi, edge/sensori/attuatori, squadre, partite, statistiche.
- Gioco: tipi di gioco, regole, modelli sensore, controllo installazioni.
- Giocatore: giochi disponibili, partite, statistiche, tornei.

L'interfaccia edge locale mostra:

- stato MQTT;
- coda offline;
- ultimo evento;
- attuatori ricevuti;
- pulsanti per simulare online e offline.

7. Validazione



Flusso di sincronizzazione offline

7.1 Test automatici

I test verificano:

- permessi dei ruoli;

- calcolo del vincitore;
- classifica torneo;
- ruolo amministratore gioco;
- validazione dei tipi di gioco;
- torneo con più locali;
- partite individuali e a squadre;
- configurazione sensori;
- compatibilità tra torneo e partita.

Comando:

```
cd backend
```

```
npm test
```

Risultato atteso:

25 test superati

0 test falliti

7.2 Controllo statico

```
npm run check
```

```
node scripts/validate-project.js
```

Il primo comando controlla la sintassi JavaScript. Il secondo controlla file obbligatori, link HTML, tabelle richieste e servizi Docker.

7.3 Test dei microservizi e di integrazione

Con tutti i container avviati si esegue:

```
node scripts/integration-test.js
```

Il test non modifica i dati. Controlla automaticamente:

- API Gateway e stato dei tre microservizi;
- raggiungibilità dell'Edge Service;
- login dei quattro ruoli;
- Catalog Service: tipi di gioco, giochi e dispositivi;
- Match Service: elenco partite;
- Tournament Service: tornei, squadre e statistiche.

7.4 Test di integrazione e demo manuale

1. avvio di tutti i container;
2. login con i quattro ruoli;
3. creazione tipo di gioco e modello sensore;
4. creazione edge, sensore e attuatore;
5. creazione squadra;
6. avvio partita individuale;
7. simulazione MQTT;
8. verifica punteggio e attuatore;

9. simulazione offline dall'interfaccia edge;
 10. sincronizzazione dopo il ritorno online;
 11. creazione torneo con due locali;
 12. collegamento della partita a un turno;
 13. verifica classifica.
-

8. Sicurezza e scelte didattiche

Per rendere il progetto facile da eseguire e spiegare:

- le password demo sono in chiaro;
- l'autenticazione usa l'header X-User-Id;
- i microservizi condividono lo stesso database;
- non è configurato HTTPS.

Queste non sono scelte consigliate per un prodotto reale. In produzione si userebbero password con hash, JWT, HTTPS, segreti esterni e database separati o ownership più rigida.

9. Conclusione

Il progetto soddisfa le funzionalità richieste: giochi connessi, quattro ruoli, sensori, attuatori, edge offline, MQTT, REST, microservizi, partite individuali e a squadre, tornei multi-locale, classifiche, statistiche, test e demo.

L'architettura rimane semplice: ogni modulo ha un compito chiaro e il flusso principale può essere spiegato seguendo un evento dal sensore fino al database e al display.

Appendice A - Casi d'uso testuali

UC1 - Login

Attore: qualsiasi utente.

Precondizione: account presente.

Flusso: inserisce username e password; il Catalog Service controlla MySQL; il sistema restituisce ruolo e locale; il browser apre la dashboard corretta.

Alternativa: credenziali errate, risposta 401.

Postcondizione: utente salvato nel local storage.

UC2 - Definire un tipo di gioco

Attore: amministratore del gioco.

Precondizione: login GAME_ADMIN.

Flusso: inserisce nome, descrizione, eventi e limite; il server valida; salva game_types; aggiunge modelli sensore.

Alternativa: nome duplicato o dati mancanti.

Postcondizione: il tipo è disponibile quando si installa un gioco.

UC3 - Installare un gioco nel locale

Attore: amministratore locale.

Precondizione: esiste un tipo di gioco.

Flusso: seleziona tipo, nome e stato; il gioco viene associato automaticamente al suo locale.

Postcondizione: il gioco compare nella lista locale e nella lista dei giocatori se online.

UC4 - Configurare edge, sensore e attuatore

Attore: amministratore locale.

Flusso: crea dispositivo edge; crea sensore scegliendo gioco ed evento; crea display o LED; il sistema verifica che edge e gioco siano nello stesso locale.

Postcondizione: configurazione salvata e visibile.

UC5 - Avviare partita individuale

Attore: amministratore locale.

Precondizione: gioco online e due client registrati.

Flusso: seleziona giocatori; il Match Service crea la partita; il gioco passa a IN_GAME.

Alternativa: giocatore non registrato o gioco offline.

Postcondizione: partita LIVE.

UC6 - Avviare partita a squadre

Attore: amministratore locale.

Precondizione: due squadre diverse e tipo che supporta squadre.

Flusso: seleziona modalita TEAM e le due squadre; il server registra i nomi delle squadre.

Postcondizione: partita TEAM LIVE.

UC7 - Ricevere eventi MQTT

Attore: edge service.

Flusso: pubblica evento con UUID; Match Service controlla partita, gioco e locale; verifica duplicato; aggiorna punteggio; salva evento; aggiorna attuatori.

Alternativa: UUID già presente, evento ignorato senza doppio punteggio.

UC8 - Funzionare offline

Attore: edge service.

Flusso: connessione assente; evento salvato in coda JSON con PENDING; al ritorno online la coda viene pubblicata; il server deduplica; la coda viene svuotata.

Postcondizione: eventi sincronizzati.

UC9 - Creare torneo multi-locale

Attore: amministratore piattaforma.

Flusso: sceglie tipo, modalita, date e almeno un locale; per TEAM sceglie squadre; salva torneo e relazioni.

Postcondizione: torneo visibile a tutti.

UC10 - Collegare partita al calendario

Attore: amministratore piattaforma o locale autorizzato.

Precondizione: partita FINISHED con stesso tipo, modalita e locale del torneo.

Flusso: sceglie turno e data prevista; salva tournament_matches.

Postcondizione: calendario e classifica aggiornati.

Appendice B - Matrice dei requisiti

Requisito docente	Implementazione	File principali	Demo
Giocatori	CLIENT	users, client pages	login client
Admin locale	LOCAL_ADMIN	user/device/game controllers	login localadmin
Admin gioco	GAME_ADMIN	gameTypeController	login gameadmin
Admin piattaforma	PLATFORM_ADMIN	locale/user/tournament controllers	login platform
Tipi e regole gioco	game_types	game-admin-dashboard	crea tipo
Sensori associati	sensor_templates + sensors	deviceController	crea sensore
Edge locale	edge-service	simulator.js	porta 8090
Offline	coda JSON	simulator/data	pulsante offline
MQTT	Mosquitto QoS 1	mqttClient.js	simula partita
Attuatori	actuators	actuatorService.js	display stato
REST	API gateway e servizi	routes + openapi	browser/Postman
Microservizi	3 servizi + gateway	docker-compose	docker ps
Partite individuali	participant_mode	matchController	start individual
Partite a squadre	teams/team_members	teams page	start TEAM
Tornei multi-locale	tournament_locations	tournaments page	seleziona locali
Calendario/turni	round_number/scheduled_at	tournament_matches	collega partita
Classifica	tournamentRules	tournament detail	vedi punti
Statistiche	statsController	dashboard	statistiche
UML e sequenze	Mermaid	docs/03 e docs/14	relazione
OpenAPI	YAML completo	docs/openapi.yaml	Swagger Editor
Test	Node test	backend/tests	npm test

Appendice C - Procedura demo

Preparazione

docker compose down -v

docker compose up --build -d

Aprire:

- piattaforma: `http://localhost:8080`
- edge locale: `http://localhost:8090`
- health API: `http://localhost:3000/api/health`

Sequenza consigliata

1. Login gameadmin / game123.
2. Mostrare tipi di gioco e modelli sensore. Creare un tipo demo.
3. Login localadmin / local123.
4. Aprire Edge e sensori; mostrare dispositivo, sensori e display.
5. Aprire Squadre; mostrare Red Lions e Blue Rockets.
6. Aprire Giochi; avviare partita individuale Mario-Luigi.
7. Nella pagina live premere simulazione MQTT.
8. Mostrare punteggio che cambia ed eventi ricevuti.
9. Aprire localhost:8090; mostrare stato attuatore.
10. Premere "Simula offline" e inviare un evento manuale: compare nella coda.
11. Premere "Vai online": la coda si svuota.
12. Login platform / platform123.
13. Mostrare utenti con i quattro ruoli.
14. Aprire Tornei; mostrare locali coinvolti, calendario e classifica.
15. Concludere mostrando npm test e la matrice requisiti.

Frase semplice per spiegare il flusso

Il sensore genera un evento, l'edge lo mette su un topic MQTT, il Match Service lo controlla e lo salva, poi aggiorna il punteggio e invia lo stato al display. Se Internet manca, l'edge conserva l'evento e lo invia dopo.